
Introduction to Parallel Programming

Lecture: MPI Collective

谭光明

高性能计算机研究中心

Collectives in MPI

Collectives: operations including all ranks of a communicator

All ranks must call the function!

- Blocking variants: buffer can be reused after return
- Nonblocking variants (since MPI 3.0):
 - buffer can be used after completion (MPI_Wait*/MPI_Test*)
- May or may not synchronize the processes
- Cannot interfere with point-to-point communication
 - Completely separate modes of operation!

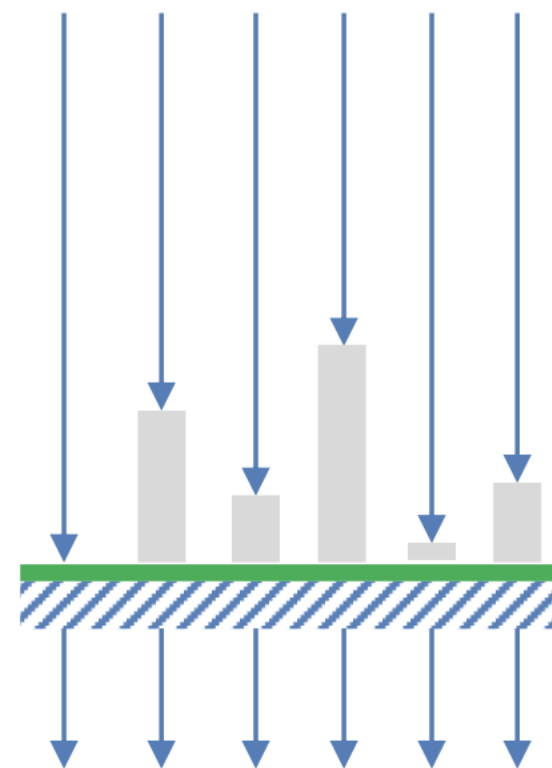
Collectives in MPI

- Rules for all collectives
 - Data type matching
 - No tags
 - Count must be exact, i.e., there is only one message length, buffer must be large enough
- Types:
 - Synchronization (barrier)
 - Data movement (broadcast, scatter, gather, all-to-all)
 - Collective computation (reduction, scan)
 - Combinations of data movement and computation (reduction + broadcast)
- General assumption: MPI does a better job at collectives than you trying to emulate them with a collection of point-to-point calls

Barrier

```
MPI_Barrier(comm);
```

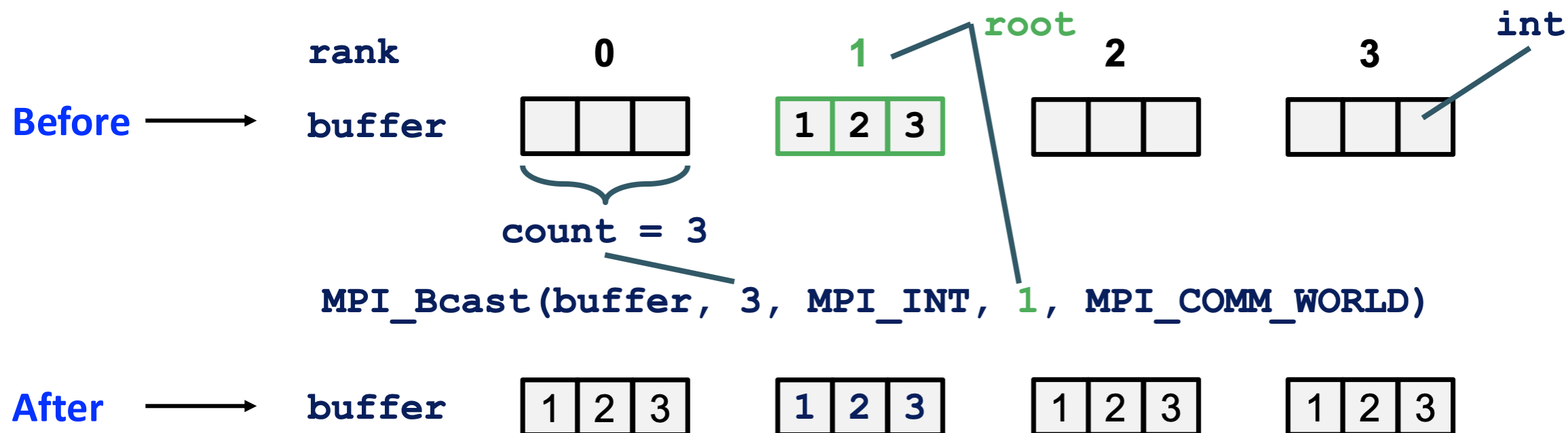
- Explicit synchronization of all ranks from specified communicator
- Ranks only return from call after every rank has called the function
- **MPI_Barrier**: rarely needed
 - Debugging



Broadcast

```
MPI_Bcast(buf, count, datatype, int root, comm);
```

- Send buffer contents from one rank ("root") to all ranks
- no restrictions on which rank is root - often rank 0

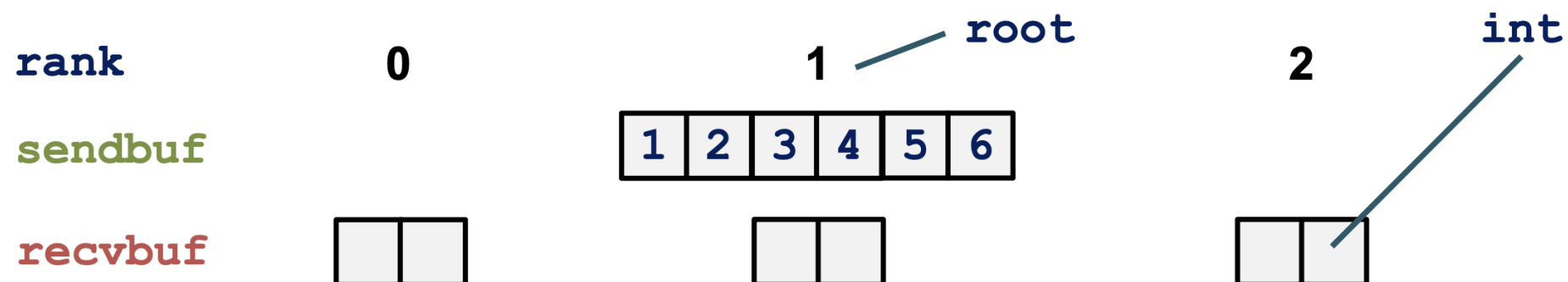


Scatter

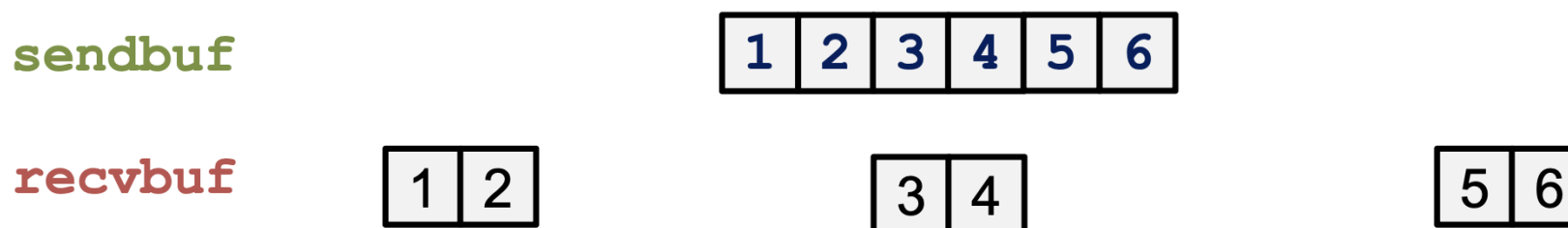
```
MPI_Scatter(sendbuf, sendcount, sendtype,  
            recvbuf, recvcount, recvtype,  
            root, comm);
```

- Send every i-th chunk of an array to the i-th rank
- Root and comm must be the same on all processes
- Type signature of send and receive variables must match
- Usually, sendcount= recvcount because sendtype = recvtype. This is the length of the chunk
- sendbuf is ignored on non-root ranks because there is nothing to send

Scatter



```
MPI_Scatter(sendbuf, 2, MPI_INT, recvbuf, 2, MPI_INT, root, MPI_COMM_WORLD);
```



Scatter

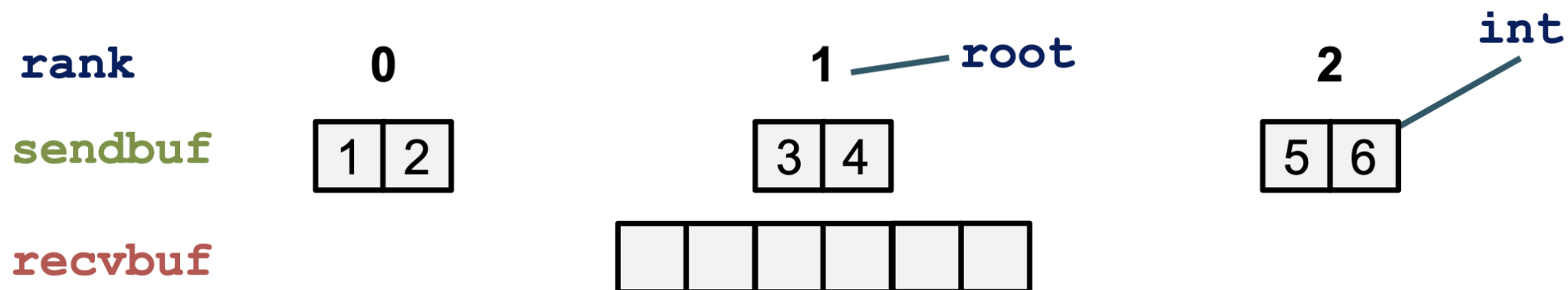
```
MPI_Comm comm;  
int np, *sendbuf;  
int root, rbuf[100];  
.....  
MPI_Comm_size(comm, &np);  
sendbuf = (int *)malloc(np * 100 * sizeof(int));  
.....  
MPI_Scatter(sendbuf, 100, MPI_INT, rbuf, 100,  
MPI_INT, root, comm);
```

Gather

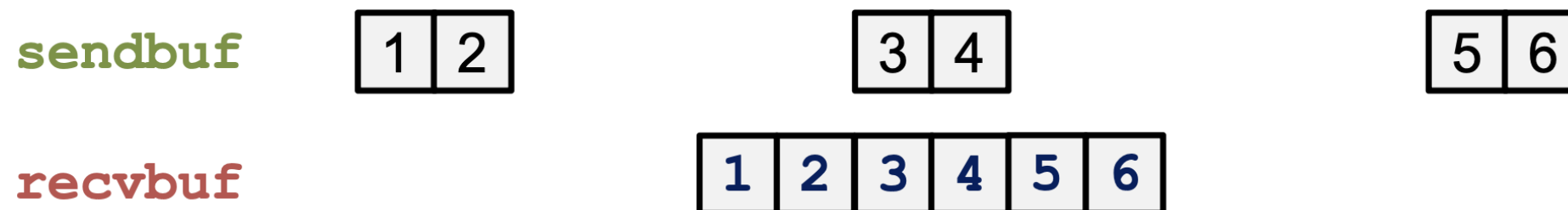
```
MPI_Gather(sendbuf, sendcount, sendtype,  
           recvbuf, recvcount, recvtype,  
           root, comm)
```

- Receive a message from each rank and place i-th rank's message at i-th position in receive buffer
- Root and comm must be the same on all processes
- Type signature of send and receive variables must match
- Usually, **sendcount=recvcount** because **sendtype = recvtype**
- **recvbuf** is ignored on non-root ranks because there is nothing to receive

Gather



```
MPI_Gather(sendbuf, 2, MPI_INT, recvbuf, 2, MPI_INT, root, MPI_COMM_WORLD)
```



Gather

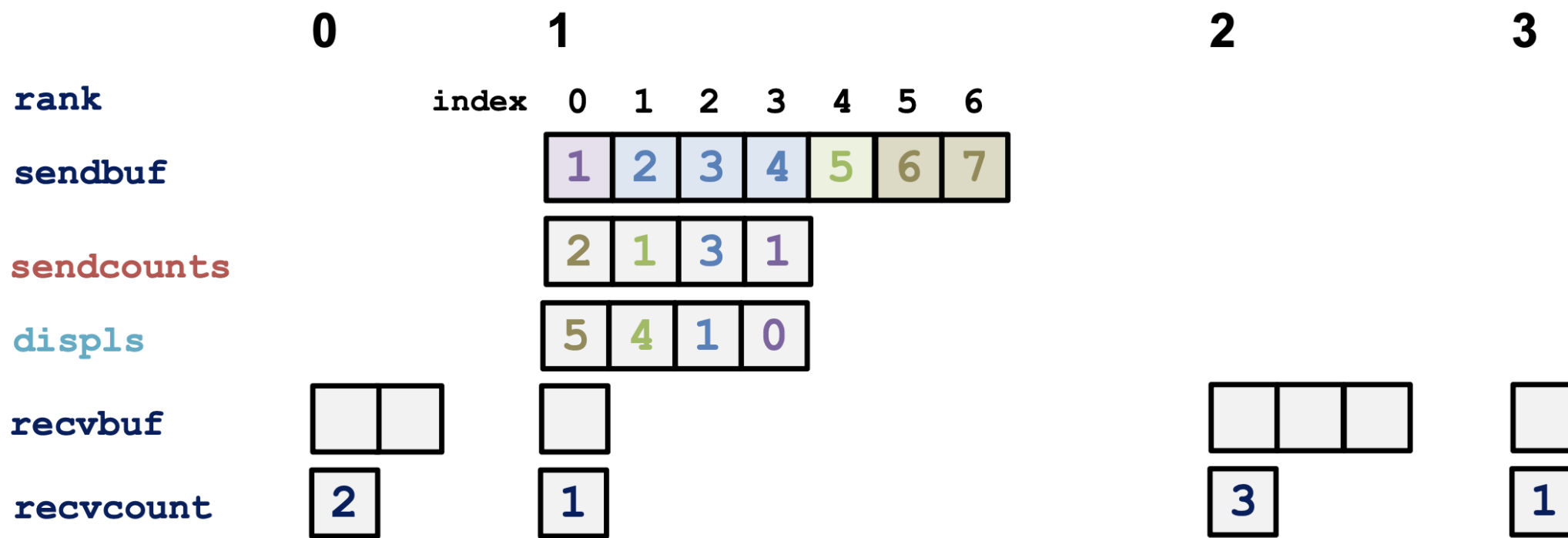
```
MPI_Comm comm;  
MPI_Comm comm;  
int      np, root, s_data[100], *rbuf;  
...  
MPI_Comm_size( comm, &np );  
rbuf = (int *) malloc( np * 100*sizeof(int) );  
MPI_Gather( s_data, 100, MPI_INT, rbuf, 100,  
            MPI_INT, root, comm );  
...  
MPI_Finalize( );
```

Scatterv: more flexible scatter

```
MPI_Scatterv(  
    sendbuf, int sendcounts[], int displs[], sendtype,  
    recvbuf, recvcount, recvtype, root, comm)
```

- Send chunks of different sizes to different ranks
- **sendcounts[]**: array specifying the number of elements to send to each rank: send **sendcounts[i]** elements to rank **i**
- **displs[]**: integer array specifying the displacements in **sendbuf** from which to take the outgoing data to each rank, specified in number of elements

Scatterv: more flexible scatter



MPI_Scatterv() with root = 1



Scatterv

```
int  i, np, root,*sendbuf, rbuf[100],*displs,*counts;
int  stride = 120;

.....
MPI_Comm_size(comm, &np);
sendbuf = (int *)malloc(np * stride * sizeof(int));
displs = (int *)malloc(np * sizeof(int));
counts = (int *)malloc(np * sizeof(int));
for (i=0; i<np; ++i) {
    displs[i] = i*stride;  counts[i] = 100;}
MPI_Scatterv(sendbuf, counts, displs, MPI_INT, rbuf,
100, MPI_INT, root, comm);
```

Gatherv: more flexible gather

```
MPI_Gatherv(  
    sendbuf, sendcount, sendtype,  
    recvbuf, int recvcnts[], int displs[], recvtype,  
    root, comm)
```

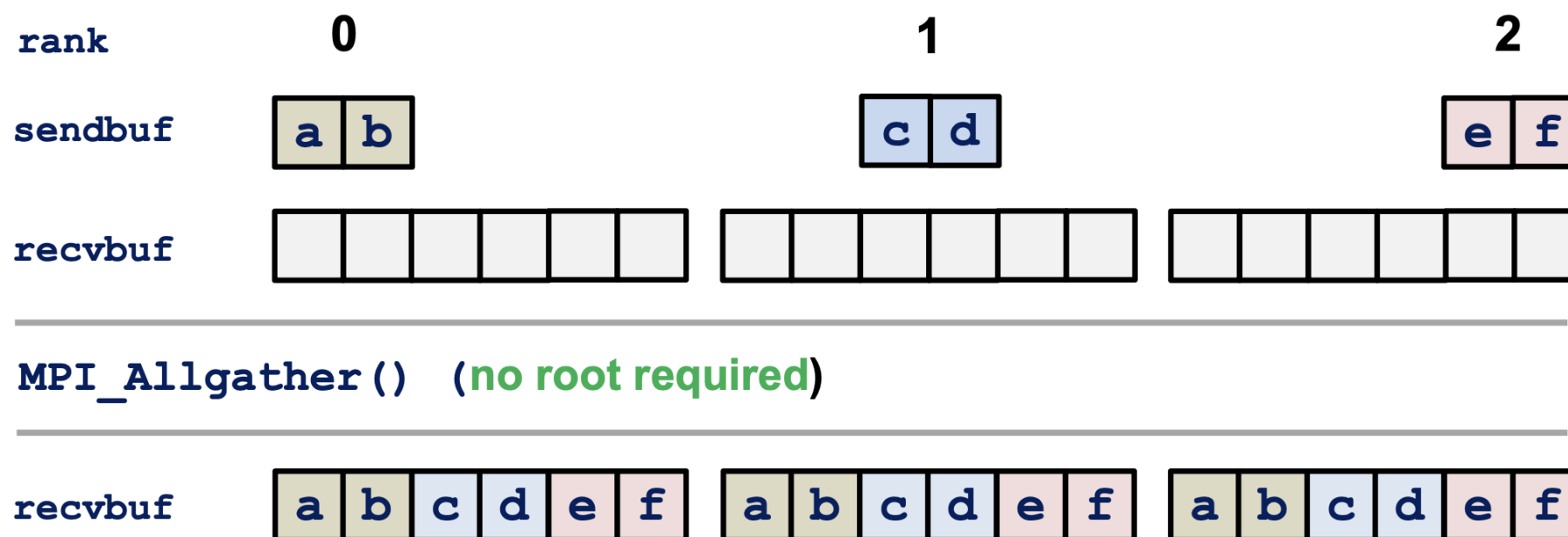
- Receive segments of different sizes from different ranks
- **recvcnts[]**: array specifying the number of elements to receive from each rank: receive **recvcnts[i]** elements from rank *i*
- **displs[]**: integer array specifying the displacements where received data from specific rank is put in **recvbuf**, in units of elements:

Gatherv

```
int      np, root, s_data[100], *rbuf, *displs, *rcount;
int      stride = 120;
...
MPI_Comm_size( comm, &np );
rbuf = (int *) malloc( np * stride * sizeof(int) );
displs = (int *) malloc( np * sizeof(int) );
rcounts = (int *) malloc( np * sizeof(int) );
for (i = 0; i < np; i ++){
    displs[i] = i * stride;  rcounts[i] = 100;}
MPI_Gatherv( s_data, 100, MPI_INT, rbuf, rcounts, displs,
MPI_INT, root, comm);
...
```

Allgather

- Combination of gather and broadcast
- Also available: **MPI_Allgatherv()** (cf. **MPI_Gatherv()**)
- Why not just use gather followed by a broadcast instead?
 - MPI library has more options for optimization
 - General assumption: Combined collectives are faster than using separate ones



In this example: sendcount=recvcount=2

Allgather

```
int MPI_Allgather (void* sendbuf, int sendcount, MPI_Datatype sendtype,  
                  void* recvbuf, int recvcount, MPI_Datatype recvtype,  
                  MPI_Comm comm)
```

```
MPI_Comm comm;  
  
int size, sendarray[100], *rbuf;  
.....  
MPI_Comm_size(comm, &np);  
  
rbuf = (int *)malloc(np * 100 * sizeof(int));  
  
MPI_Allgather(sendarray, 100, MPI_INT,  
              rbuf, 100, MPI_INT, comm);
```

Allgatherv

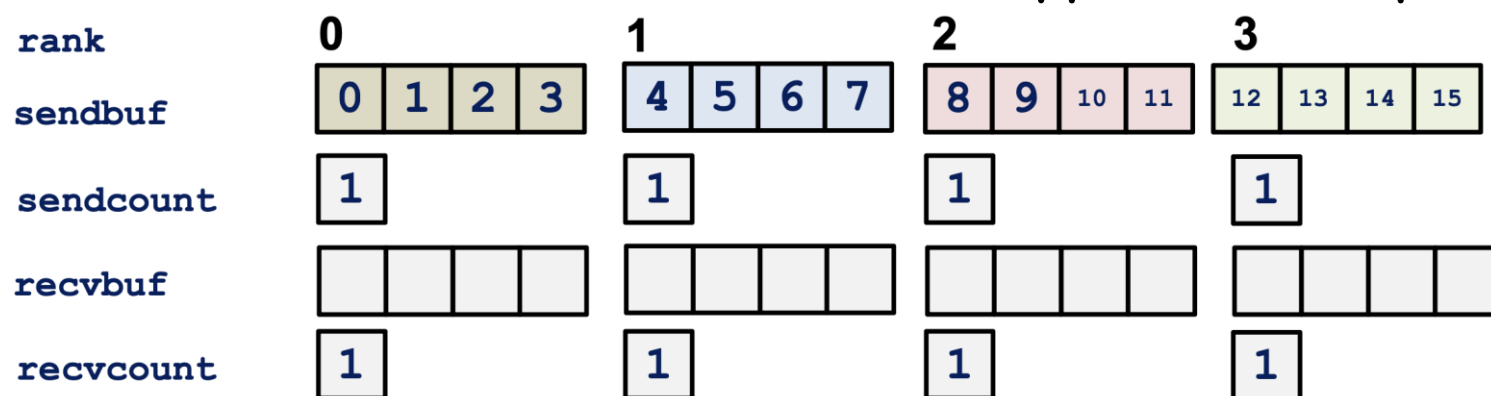
```
int MPI_Allgatherv (void* sendbuf, int sendcount, MPI_Datatype sendtype,  
                   void* recvbuf, int recvcount[], int displs[],  
                   MPI_Datatype recvtype, MPI_Comm comm)
```

```
int size, sendarray[100], *rbuf, *displs, i, *rcounts;  
int stride = 120;  
.....  
MPI_Comm_size(comm, &np);  
rbuf = (int *) malloc(np * stride * sizeof(int));  
displs = (int *) malloc(np * sizeof(int));  
rcounts = (int *) malloc(np * sizeof(int));  
for (i=0; i<np; ++i) {  
    displs[i] = i * stride;  rcounts[i] = 100;}  
MPI_Allgatherv(sendarray, 100, MPI_INT,  
               rbuf, rcounts, displs, MPI_INT, comm);
```

Alltoall

```
MPI_Alltoall(sendbuf, sendcount, sendtype,
             recvbuf, recvcount, recvtype,
             comm)
```

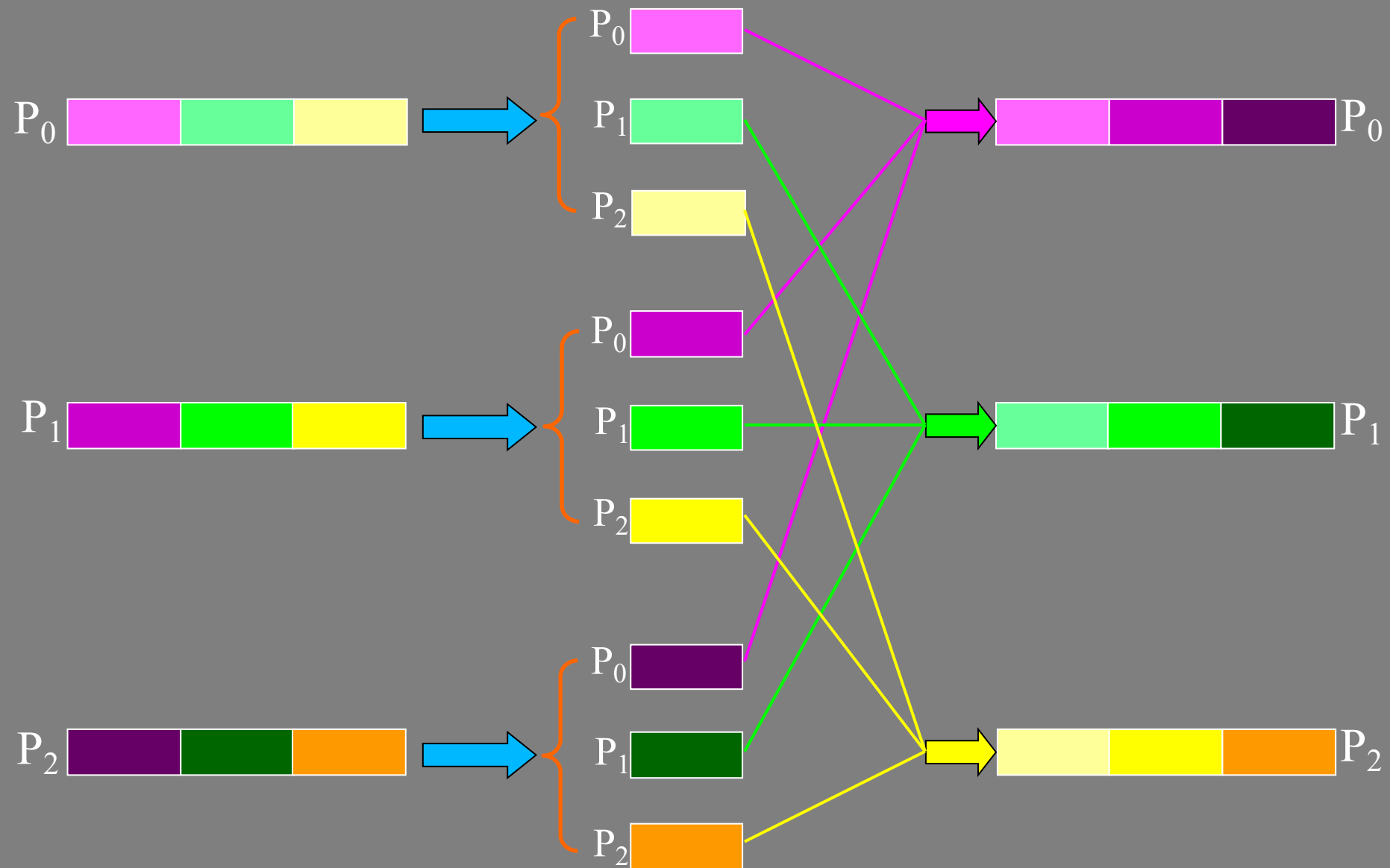
- **MPI_Alltoall**: For all ranks, send i-th chunk to i-th rank
- **MPI_Alltoallv**: Allows different number of elements to be send/received by each rank
- **MPI_Alltoallw**: Allows also different data types and displacements in bytes



MPI_Alltoall() (no root required)



Altoall



Quiz

- Why should one use collective communication rather than emulating by a set of point-to-point calls?
 - Answer: Implementations are optimized for efficiency, and are also adapted to the hardware environment at hand (not guaranteed, however). You don't want to redo the work of countless MPI researchers.
- Can MPI collective communications interfere with point-to-point calls?
 - Answer: No, because they are completely separate modes of operation.
- For a collective communication, it is not necessary every process of a communicator to call it?
 - Answer: It is necessary.

Quiz

- To send an identical piece of data to all other processes in a communicator, which collective call should be used?
 - a. `MPI_Gather`
 - b. `MPI_Bcast`
 - c. `MPI_Scatter`
 - d. `MPI_Alltoall`
 - Answer: b
- Which of the following collective calls is similar to the process of transposing a matrix in mathematics?
 - a. `MPI_Gather`
 - b. `MPI_Bcast`
 - c. `MPI_Scatter`
 - d. `MPI_Alltoall`
 - Answer: d

Exercise: SendRecvCollectives/ GatherScatter

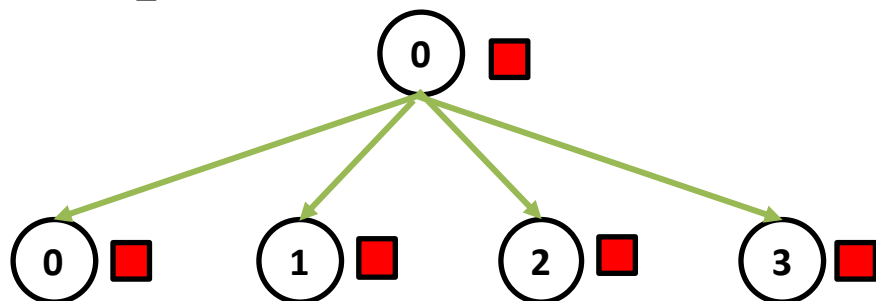
- Generate a random array of numbers on the root process (process 0).
- Scatter the numbers to all processes, giving each process an equal amount of numbers.
- Each process computes the average of their subset of the numbers.
- Gather all averages to the root process. The root process then computes the average of these numbers to get the final average.

Directory SendRecvCollectives: Implement

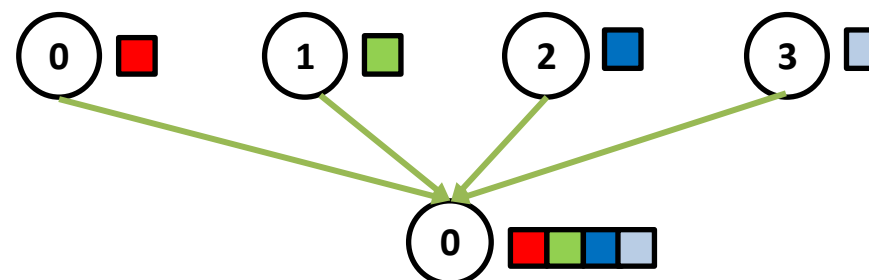
- call to `MPI_Scatter(...)`
- call to `MPI_Gather(...)`

Exercise: GatherScatter implemented in MPI

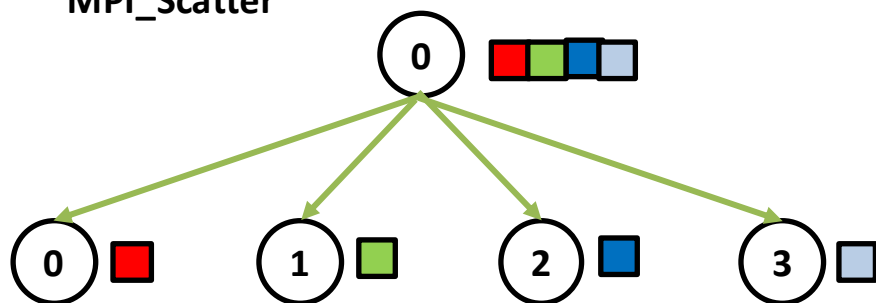
MPI_Bcast



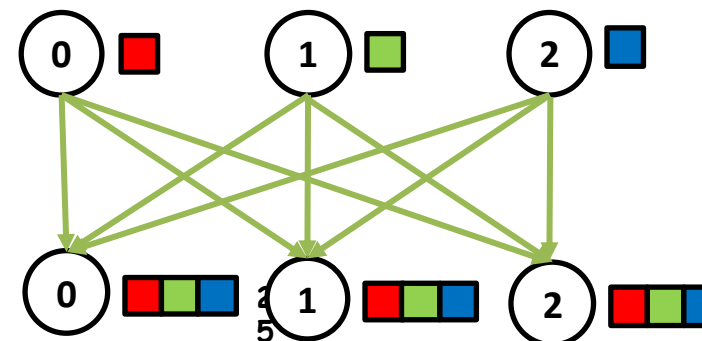
MPI_Gather



MPI_Scatter



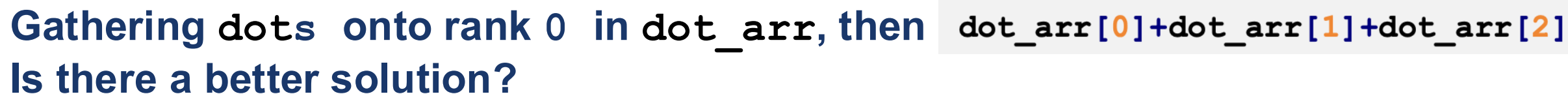
MPI_Allgather



- Dot product: basic reduction using MPI collectives

- `dot=0.0;`
`for(int i=0;i<N;i++) dot+=a[i]*b[i];`

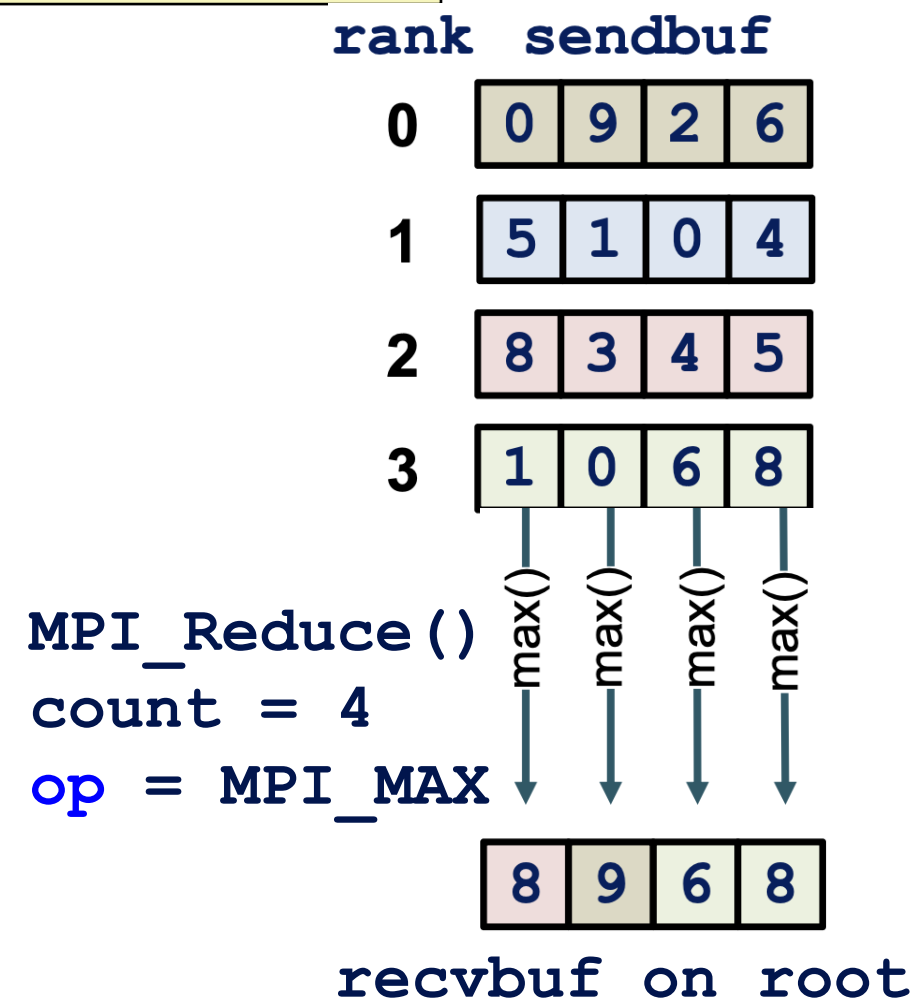
$$\xrightarrow{N_l=N/N_{procs}}$$
`dot=0.0;`
`for(int i=0;i<Nl;i++) dot+=a[i]*b[i];`



Global operations: reduction

```
MPI_Reduce(sendbuf, recvbuf, count,
           datatype, MPI_Op op, root, comm)
```

- Compute results over distributed data
- Result in **recvbuf** only available on root process
- Perform operation on all **count** elements of an array
- If all ranks need the result, then use **MPI_Allreduce()** (NO root)
- If the 12 predefined ops are not enough use **MPI_Op_create/MPI_Op_free** to create own ones



Global operations – predefined operators

Name	Operation	Name	Operation
MPI_SUM	Sum	MPI_PROD	Product
MPI_MAX	Maximum	MPI_MIN	Minimum
MPI_LAND	Logical AND	MPI_BAND	Bit-AND
MPI_LOR	Logical OR	MPI_BOR	Bit-OR
MPI_LXOR	Logical XOR	MPI_BXOR	Bit-XOR
MPI_MAXLOC	Maximum+Position	MPI_MINLOC	Minimum+Position

- Define own operations with **MPI_Op_create/MPI_Op_free**
- MPI assumes that the operations are associative
 - → be careful with floating-point operations

“In-place” buffer specification

- Override local input buffer with a result

MPI_Reduce

```
int partial_sum = ..., total_sum;

MPI_Reduce(&partial_sum, &total_sum, 1,
MPI_INT, MPI_SUM, root, comm)

int partial_sum = ..., total_sum;
if (rank == root) {
    total_sum = partial_sum;
    MPI_Reduce(MPI_IN_PLACE, &total_sum,
1, MPI_INT, MPI_SUM, root, comm);
} else {
    MPI_Reduce(&partial_sum, &total_sum,
1, MPI_INT, MPI_SUM, root, comm)
}
```

MPI_Allreduce

```
int partial_sum = ..., total_sum;

MPI_Reduce(&partial_sum,
&total_sum, 1, MPI_INT, MPI_SUM,
root, comm)

int partial_sum = ..., total_sum;

total_sum = partial_sum;
MPI_Allreduce(MPI_IN_PLACE,
&total_sum, 1, MPI_INT, MPI_SUM,
root, comm)
```

Nonblocking Collectives in MPI

Similar to blocking collectives: nonblocking collective calls
including all ranks of a communicator
All ranks must call the function!

- Nonblocking variants (since MPI 3.0):
- buffer can be used after completion (`MPI_Wait*`/`MPI_Test*`)
- Local: not synchronization
- Multiple outstanding collective communications on same communicator
- Cannot interfere with point-to-point communication
 - Completely separate modes of operation!
- Cannot interfere with blocking collective communication
 - Such interference was allowed in point-to-point communication

Collectives: Blocking vs. Nonblocking

- Barrier
 - `MPI_barrier(MPI_Comm comm)`
 - `MPI_Ibarrier(MPI_Comm comm, MPI_Request *request)`
- Broadcast:
 - `MPI_Bcast(buf, count, datatype, root, comm);`
 - `MPI_Ibcast(buf, count, datatype, root, comm, request);`
- Scatter:
 - `MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm);`
 - `MPI_Iscatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm, request);`
- Gather:
 - `MPI_Gather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm);`
 - `MPI_Igather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm, request);`
- Similarly many blocking collective calls have nonblocking analogues

Summary of MPI Collective Communications

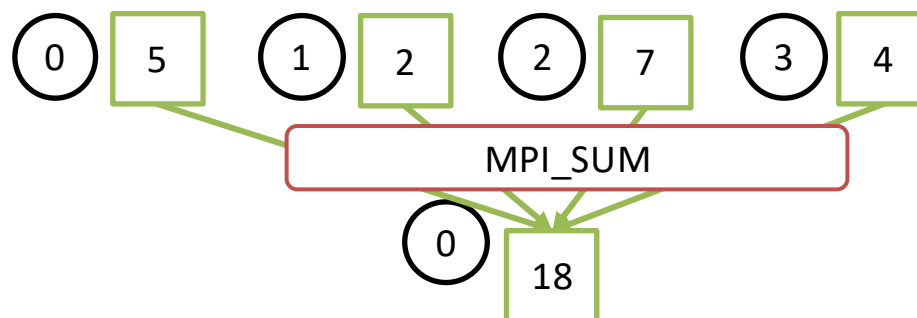
- MPI (blocking) collectives
 - All ranks in communicator must call the function
- Communication and synchronization
 - Barrier, broadcast, scatter, gather, and combinations thereof
- Global operations
 - Reduce, allreduce, some more...
- In-place buffer specification **MPI_IN_PLACE**
 - Save some space if needed

Quiz

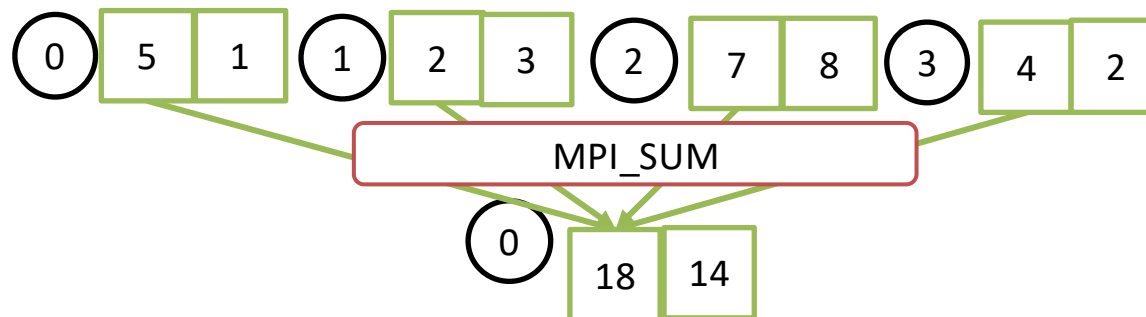
- Do you recommend an application developer to use **MPI_Gather** followed by operations on the **root** process rather than using **MPI_Reduce**? Why?
 - Answer: No, because it is very likely that it is slower than **MPI_Reduce**.
- Is the argument **recvbuf** in **MPI_Reduce** significant on every process calling it?
 - Answer: No, it is significant only at root.

Exercise: Reduce

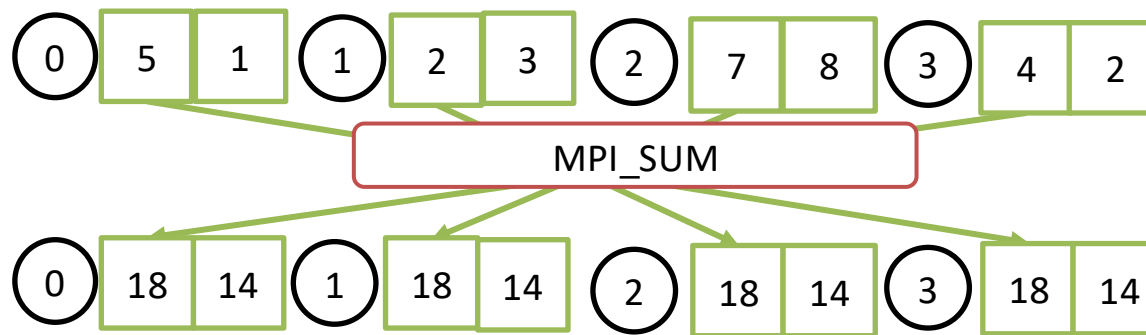
MPI_Reduce



MPI_Reduce

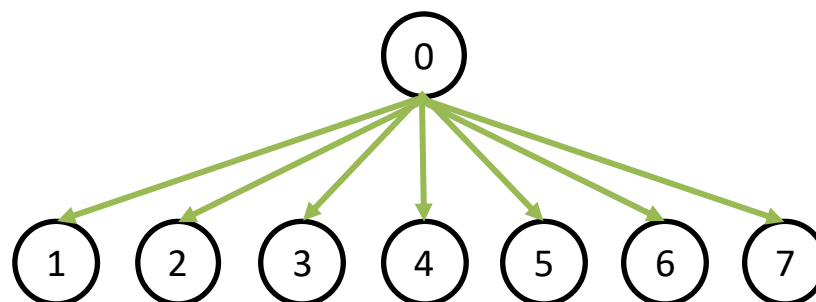


MPI_Allreduce

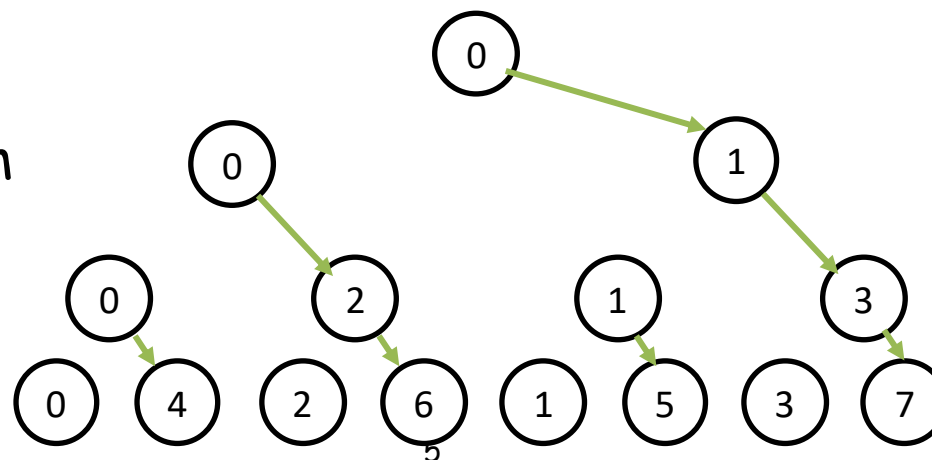


Exercise: SendRecvCollective/ BroadcastBarrier

‘Write’ and compare a simple broadcast program,



with a more advanced algorithm



Exercise: Matrix-Vector

